A7 2D Arrays, Pathfinding & Game Algorithms

Modern Code
MC (MTD.ba VZ)
SS 2025

Submission via Moodle — **24 Points**

This assignment focuses on 2D arrays, pathfinding algorithms, and game AI through implementing a complete Treasure Collection game. You will create a game where players move on a grid collecting treasures, using pathfinding algorithms (DFS, BFS, A*, or any other) to find optimal routes and MinMax algorithm for AI decision-making.

Game Overview:

- Configurable grid size with treasures (coins) and obstacles (walls)
- Player and AI take turns moving one step (up/down/left/right)
- Collect treasures by moving onto them
- Cannot move through obstacles or occupied cells
- Game ends when all treasures collected or no valid moves remain
- Winner: Player who collected more treasures

Submission Requirements:

- Read the `Organizational Guidelines on Moodle`, and adhere to them!
- Submit a PDF document containing your **solution idea, code, and tests for each exercise**
- Include all Java source files (.java) in a ZIP archive
- As tests include copies or screenshots of the console output
- Use the naming convention: `MC_UE07_LASTNAME.pdf` and `MC_UE07_LASTNAME.zip`

Important: This assignment emphasizes understanding pathfinding algorithms and game AI. For each exercise, think about:

- How to represent the game board using 2D arrays
- How to use pathfinding algorithms (DFS, BFS, A*, or any other) to find paths to treasures
- How to implement MinMax for AI decision-making
- How to evaluate game positions and handle depth limiting

Important: This is a single assignment implementing one complete Treasure Collection game. The exercises below show how points are distributed across different aspects of the implementation. Think about:

- How to represent the game board using 2D arrays
- How to use BFS pathfinding to find shortest paths to treasures
- How to implement MinMax for optimal AI decision-making
- How to structure your code for clarity and maintainability
- How to handle edge cases (no treasures, unreachable treasures, game end conditions)

Exercise 1 (4 points) Game Board Setup & Display

Create the game board representation and display functionality. Your program should be called `TreasureCollectionGame.java`.

1. Create a 2D array to represent the game board (grid size should be configurable, e.g., through method parameters or constants)
2. It should have the following: empty cell, player position, AI position, treasure, obstacle (wall)
3. Initialize the board with:
   - Player starting position (e.g., top-left corner)
   - AI starting position (e.g., bottom-right corner)
   - Several treasures scattered across the board (3-5 treasures)
   - Some obstacles blocking paths (2-3 obstacles)
4. Implement a method to display the board with clear visual representation
5. Test with different board configurations

Expected Output:

```
=== Treasure Collection Game ===
Initial Board:
  0 1 2 3 4
0 P . . . .
1 . # . T .
2 . . . . .
3 . T . # .
4 . . . . A

Player: 0 treasures | AI: 0 treasures
```

(Where P = Player, A = AI, T = Treasure, # = Obstacle, . = Empty)

Exercise 2 (6 points) Player Movement & Pathfinding

Implement player movement and use pathfinding for the AI to find the shortest path to the nearest treasure.

1. Implement player movement (up/down/left/right) with validation
2. Check that moves are valid (within bounds, not obstacle, not occupied)
3. Use a pathfinding algorithm (DFS, BFS, A*, or any other) to find a path from current position to nearest treasure
4. Display the path found (show the sequence of moves or path length)
5. Handle edge cases (no treasures remaining, unreachable treasures)

Expected Output:

```
=== Player Movement & Pathfinding ===
Current position: [0][0]
Nearest treasure at: [1][3]
Shortest path length: 4 steps
Path: Right   Right   Right   Down

After moving right:
  0 1 2 3 4
0 . P . . .
1 . # . T .
2 . . . . .
3 . T . # .
4 . . . . A

Player: 0 treasures | AI: 0 treasures
```

Exercise 3 (4 points) Simple Greedy AI

Implement a simple greedy AI that moves toward the nearest treasure using pathfinding.

1. Create a greedy AI that uses pathfinding to find the nearest treasure
2. AI should move one step toward the nearest treasure each turn
3. Handle cases where multiple treasures are equidistant (choose one)
4. Display AI's decision and movement
5. Compare AI behavior to player movement

Expected Output:

```
=== Simple Greedy AI ===
AI current position: [4][4]
AI found nearest treasure at: [1][3]
AI moving: Up   Up    Up    Left
AI moved: Up

Board after AI move:
  0 1 2 3 4
0 P . . . .
1 . # . T .
2 . . . . .
3 . T . # .
4 . . . . A
```

Exercise 4 (6 points) MinMax AI Implementation

Implement MinMax algorithm for the AI opponent to make optimal strategic decisions.

1. Implement MinMax algorithm with depth limiting (recommended: 3-4 moves ahead)
2. Create an evaluation function that calculates treasure count difference: player_treasures - AI_treasures
3. Handle game tree exploration (each player has up to 4 possible moves per turn)
4. AI should choose the move that maximizes its minimum guaranteed outcome
5. Compare MinMax AI performance vs greedy AI

Expected Output:

```
=== MinMax AI ===
AI evaluating moves with MinMax (depth: 3)...
AI chose move: Up
MinMax score: +1 (AI advantage)

Board after MinMax AI move (was 4,4 is 3,4):
  0 1 2 3 4
0 P . . . .
1 . # . T .
2 . . . . .
3 . T . # A
4 . . . . .
```

Exercise 5 (4 points) Complete Game Integration

Integrate all components into a complete, playable game.

1. Implement game loop (player turn AI turn repeat)
2. Add win/lose detection (all treasures collected or no valid moves)
3. Display game statistics (treasures collected, moves taken, current scores)
4. Handle game end and declare winner
5. Test with different scenarios (player wins, AI wins, obstacles blocking paths)

Expected Output:

```
=== Complete Treasure Collection Game ===
Turn 1:
Player turn - Choose direction (U/D/L/R): R
Player moved right and collected a treasure!
Player: 1 treasures | AI: 0 treasures

AI turn - MinMax choosing move...
AI moved up.

Turn 2:
...
========================================
           GAME OVER
========================================
All treasures collected!
Final Scores:
  Player: 2 treasures
  AI: 1 treasure
Player wins!
```
